

XQuery

Introducción

XQuery es un **lenguaje declarativo** para **consultar** y **transformar** datos XML. Comparte modelo de datos con XPath y cumple un papel similar al de SQL en el mundo relacional.

A nivel semántico, tiene similitudes con el **lenguaje SQL**, aunque incluye algunas capacidades de programación a mayores. XQuery permite la construcción de expresiones complejas combinando expresiones simples de una manera muy flexible. También comparte similitudes con XPath, con el cual comparte modelo de datos y soporta las mismas funciones y operadores.

De manera general, podemos decir que XQuery es a XML lo mismo que SQL es a las bases de datos relacionales. Al igual que éste último, XQuery es un lenguaje funcional.

Aplicaciones

Los principales usos de XQuery se resumen en tres:

- Recuperar información a partir de conjuntos de datos XML.
- Transformar unas estructuras de datos XML en otras estructuras que organizan la información de forma diferente.
- Ofrecer una alternativa a XSLT para realizar transformaciones de datos en XML a otro tipo de formatos, como HTML o PDF.

Requerimientos técnicos

Los requerimientos técnicos más importantes de XQuery se detallan a continuación:

- Debe ser un **lenguaje declarativo**. Esto significa que el usuario debe expresar lo que quiere obtener sin necesidad de especificar cómo se obtiene. El motor de ejecución se encarga de optimizar la consulta y determinar la mejor estrategia para obtener el resultado.
- Debe ser **independiente del protocolo de acceso a la colección de datos**. Esto significa que una consulta en XQuery debe funcionar de manera idéntica en cualquiera de los siguientes casos:
 - Al consultar un fichero XML local.
 - Al consultar un fichero XML en un servidor web.
 - Al consultar un servidor de bases de datos.
- Las consultas y los resultados deben respetar el **modelo de datos XML**.
- Las consultas y los resultados deben ofrecer **soporte para los namespaces**.

- Debe soportar XML Schemas (XSD) y DTDs y también debe ser capaz de trabajar sin ellos.
- Ha de ser independiente de la estructura del documento, esto es, funcionar sin conocerla.
- Debe soportar tipos **simples**, como **enteros** y **cadenas**, y tipos **complejos**, como un **nodo compuesto**.
- Las consultas deben soportar cuantificadores universales (para todo) y existenciales (existe).
- Las consultas deben soportar operaciones sobre jerarquías de nodos y secuencias de nodos.
- Debe ser posible combinar información de múltiples fuentes en una consulta.
- Las consultas deben ser capaces de manipular los datos independientemente del origen de estos.
- El lenguaje de consulta debe ser independiente de la sintaxis, esto es, pueden existir varias sintaxis distintas para expresar una misma consulta en XQuery.

Modelo de datos

XQuery trabaja con **secuencias** de ítems (nodos y valores atómicos). El **orden de documento** es significativo.

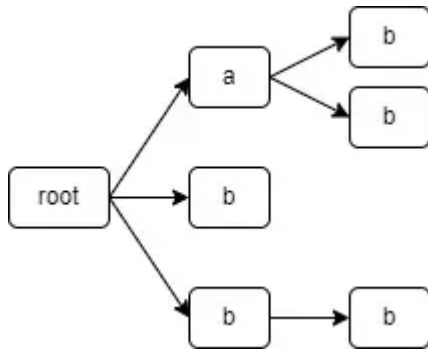
Por ejemplo, a diferencia de SQL, en XQuery el orden en que se encuentren los datos es importante, ya que no es lo mismo buscar una etiqueta `<b>` dentro de una etiqueta `<a>` que todas las etiquetas `<b>` del documento (que pueden estar anidadas dentro de una etiqueta `<a>` o no).

En el siguiente ejemplo, se muestran los siguientes casos:

- Etiquetas `<b>` dentro de una etiqueta `<a>`.
- Etiquetas `<b>` fuera de `<a>`.
- Una etiqueta `<b>` dentro de otra etiqueta `<b>`.

```
<root>
  <a>
    <b/>
    <b/>
  </a>
  <b/>
  <b>
    <b/>
  </b>
</root>
```





Características

La entrada y la salida de una consulta XQuery se define en términos de un modelo de datos. Dicho modelo de datos de la consulta proporciona una representación abstracta de uno o más documentos XML (o fragmentos de documentos).

Las principales características de este modelo de datos son:

- Se basa en la **definición de secuencia**, como una **colección ordenada de cero o más ítems**. Éstas pueden ser **heterogéneas**, es decir, pueden contener **varios tipos de nodos y valores atómicos**. Sin embargo, una secuencia nunca puede ser un ítem de otra secuencia.
- **Orden del documento**: corresponde al orden en que los nodos aparecerían si la jerarquía de nodos fuese representada en formato XML. Si el primer carácter de un nodo ocurre antes que el primer carácter de otro nodo, lo precederá también en el orden del documento.
- Contempla un valor especial llamado «**error value**», que es el resultado de evaluar una expresión que contiene un error.

Expresiones XQuery

Una consulta en XQuery es una expresión que lee una secuencia de datos en XML y devuelve otra secuencia de datos en XML como resultado.

Características

Algunas características de las expresiones XQuery son:

- En XQuery todo es una expresión que se evalúa a un valor.
- El valor de una expresión es una secuencia heterogénea de nodos y valores atómicos.
- La mayoría de las expresiones están compuestas por la combinación de expresiones más simples unidas mediante operadores y palabras reservadas.
- XPath es un lenguaje declarativo para la localización de nodos y fragmentos de información en documentos XML. Puesto que XQuery ha sido construido sobre la base de XPath y realiza la

selección de información y la iteración a través del conjunto de datos basándose en dicho lenguaje, toda expresión XPath también es una consulta XQuery válida.

Sintaxis

Aspectos básicos:

- Las expresiones evaluables dentro de XML generado se delimitan con `{ ... }`.
- Los comentarios se escriben como `(: comentario :)`.
- Admite condicionales `if ... then ... else`.
- Las consultas complejas suelen usar cláusulas **FLWOR**.

Ejemplo:

```
(: Selecciona títulos de libros de O'Reilly :)  
<ul>{  
  for $x in doc("libros.xml")/biblioteca/libros/libro  
  where $x/editorial = "O'Reilly"  
  order by $x/titulo  
  return <li>{data($x/titulo)}</li>  
}</ul>
```



Cláusulas FLWOR

Las consultas XQuery pueden estar formadas por hasta 5 tipos de cláusulas diferentes, denominadas **FLWOR**. FLWOR son las siglas de: `for`, `let`, `where`, `order by`, `return`, equivalentes a las cláusulas `SELECT`, `FROM`, `WHERE`, `ORDER BY` y `RETURN` de SQL, respectivamente.

Orden de aparición:

1. `for` (o `let`, al menos una de ambas)
2. `let`
3. `where`
4. `order by`
5. `return`



CLAUSULAS CASE SENSITIVE

Las cláusulas FLWOR son **case sensitive**, es decir, deben escribirse exactamente como se muestra (en minúsculas). No se pueden escribir en mayúsculas o con una combinación de mayúsculas y minúsculas.

for

Asocia una o más variables con cada nodo que encuentre en la colección de datos. Si en la consulta aparece más de una cláusula `for` (o más de una variable en una cláusula `for`), el resultado es el producto cartesiano de dichas variables.

La cláusula `for` es una de las más importantes de XQuery, ya que permite especificar las variables y los valores que se van a procesar en una consulta. Esta cláusula se utiliza para establecer el contexto de la consulta y para definir las expresiones que se van a evaluar. Dicho de otro modo, la cláusula `for` es la que permite establecer el ámbito y los datos a procesar en una consulta XQuery.

La sintaxis de la cláusula `for` es la siguiente:

```
for $variable in expression
```



donde:

- `$variable` es el nombre de la variable que se va a utilizar.
- `expression` es la expresión que se va a evaluar. La expresión puede ser cualquier cosa que devuelva un valor, como una cadena de texto, un número, una secuencia de nodos, etc.

Consideremos la siguiente consulta:

```
for $p in /libros/libro  
return $p/titulo
```



En este ejemplo, se utiliza la cláusula `for` para establecer el ámbito de la consulta en el elemento `libro` de la secuencia de nodos `libros`. La expresión `return $p/titulo` se utiliza para devolver el título de cada libro.

Consideremos, ahora, la siguiente consulta:

```
for $p in /empleados/empleado[apellido="García"]  
return $p/nombre
```



En este ejemplo, se utiliza la cláusula `for` para establecer el ámbito de la consulta en los empleados con el apellido García. La expresión `return $p/nombre` se utiliza para devolver el nombre de cada empleado con ese apellido.

`let`

La cláusula `let` permite **asignar un valor a una variable**, la cual se puede utilizar posteriormente. Su uso permite que las consultas sean más legibles y fáciles de mantener.

La sintaxis de la cláusula `let` es la siguiente:

```
let $variable := expression
```



donde:

- `$variable` es el nombre de la variable que se va a utilizar.
- `expression` es la expresión que se va a evaluar para asignarle un valor a la variable.

VALORES INMUTABLES

Una característica a tener en cuenta es que las variables en XQuery son **inmutables**, es decir, una vez definidas no se pueden modificar. Es decir, a pesar de denominarse *variables*, su comportamiento es idéntico al de las constantes de los lenguajes de programación.

Consideremos la siguiente consulta:

```
let $a := 5
let $b := 10
return $a + $b
```



En este ejemplo, se utiliza la cláusula `let` para asignar los valores 5 y 10 a las variables `$a` y `$b`, respectivamente. Luego, se utiliza la expresión `return $a + $b` para devolver la suma de ambas variables.

Funciones de agregación

La cláusula `let` junto con una expresión XPath adecuada permiten obtener un único valor utilizando las funciones de agregación (recuento, suma, media, etc.).

Consideremos, ahora, la siguiente consulta:

```
let $nombres := /empleados/empleado[edad>=18]/nombre
return count($nombres)
```



En este último ejemplo, se utiliza:

- La cláusula `let` para asignar los nombres de los empleados de la secuencia de nodos `empleados` a la variable `$nombres`.
- La cláusula `return` junto con la función `count()` para obtener la cantidad de nombres.

De forma alternativa, se puede realizar la siguiente consulta:

```
let $nombres := (
  for $e in /empleados/empleado
  where $e/edad >= 18
  return $e/nombre
)
return count($nombres)
```



El resultado de la consulta sigue siendo el mismo.

`where`

La cláusula `where` se utiliza para filtrar los resultados producidos por las cláusulas `for` y `let` y limitarlos a aquellos elementos que cumplen ciertas condiciones especificadas en la consulta.

La sintaxis básica de la cláusula `where` es la siguiente:

```
for $variable in //elemento
where condition
return result
```



donde:

- `$variable` representa el elemento que se va a buscar.
- `condition` es la **expresión booleana** que se evalúa para determinar si se incluye o no un elemento en el resultado.
- `result` es la información que se desea recuperar.

Utilizando where con for

Cuando utilizamos `where` con un `for`, el `where` está recibiendo una lista de elementos, los cuales recorreremos de forma individual. De esta forma, se evaluará la condición definida en el `where` a **cada uno de los elementos** que se han obtenido de la expresión de la cláusula `for`.

Consideremos la siguiente consulta:

```
for $libro in //libro
where $libro/precio > 50
return $libro/titulo
```



En este ejemplo, se utiliza:

- La cláusula `for` para seleccionar todos los elementos `libro` del documento mediante la expresión XPath `//libro`.
- La cláusula `where` para filtrar los libros cuyo precio es mayor a 50 euros. La expresión booleana `$libro/precio > 50` evalúa si el precio de cada libro es mayor a 50.
- La cláusula `return` se utiliza para recuperar el título de cada libro que cumple la condición evaluada en el `where`.

Consideremos, ahora, la siguiente consulta:

```
for $estudiante in //estudiante
where avg($estudiante/calificaciones/calificación) > 8.5
return $estudiante/nombre
```



En este ejemplo, se utiliza:

- La cláusula `for` para seleccionar todos los elementos `estudiante` del documento mediante la expresión XPath `//estudiante`.
- La cláusula `where` para filtrar los estudiantes cuyo promedio es mayor a 8.5. La expresión booleana indicada a continuación del `where` calcula el promedio de calificaciones de cada estudiante usando la función `avg()` y evalúa si es mayor a 8.5.
- La cláusula `return` se utiliza para recuperar el nombre de cada estudiante que cumple la condición.

Utilizando where con let

Cuando utilizamos `where` con un `let`, el `where` **está recibiendo un único elemento**. Teniendo esto en cuenta, se evaluará la condición definida en el `where` a **toda la variable**.

Consideremos la siguiente consulta:

```
let $n := 2
where $n > 1
return $n
```



En este ejemplo, se utiliza:

- La cláusula `let` para declarar una variable `$n` que guarda el valor 2.
- La cláusula `where` para filtrar si el valor de `$n` es mayor a 1.
- La cláusula `return` para recuperar el valor de `$n`.

En este caso, el resultado de la consulta sería 2.

Ahora, consideremos esta consulta:

```
let $n := 2
where $n < 1
return $n
```



Se ha modificado la expresión booleana dentro del `where`. En este caso, al no cumplirse, **no se devuelve nada**.

No es posible utilizar `where` junto con `let` de la misma forma que se utiliza con `for` cuando trabajamos con una lista de elementos.

Consideremos la siguiente consulta:

```
let $a := (1,2,3)
where $a>2
return $a
```



En este ejemplo, se podría pensar que se mostrarían aquellos elementos mayores a 2, pero no ocurre así: **se devuelven todos los elementos** porque ese es el valor de la variable `$a`. Esto ocurre así porque el valor de la variable `$a` se trata como un único valor.

Por último, consideremos la siguiente consulta:

```
let $nums := (1,2,3)
where count($nums) > 3
return $nums
```



En este caso, en el `where` se están contando la cantidad de elementos que tiene la lista definida en `$nums` mediante la función `count()`. Pueden ocurrir dos situaciones:

- Si el recuento es mayor a 3, se muestra el valor de la variable `$nums`.
- Si el recuento es 3 o menos, no se muestra nada.

En este caso, ocurre lo último.

`order by`

La cláusula `order by` se utiliza para ordenar los resultados de una consulta en función de ciertos criterios especificados en la consulta. Ordena el resultado generado por `for` y `let` después de que han sido filtradas por la cláusula `where`.

La sintaxis básica de la cláusula `order by` es la siguiente:

```
for $variable in //elemento
where condition
order by criteria1, criteria2, ... criteriaN descending
return result
```



donde:

- `$variable` representa el elemento que se va a buscar.
- `condition` es la expresión booleana que se evalúa para determinar si se incluye o no un elemento en el resultado.
- Los criterios son los campos por los cuales se desea ordenar los resultados. Por defecto, el orden es **ascendente**, pero se puede añadir el modificador `descending` para ordenar por orden descendiente.
- `result` es la información que se desea recuperar.

Consideremos la siguiente consulta:

```
for $libro in //libro
order by $libro/autor, $libro/título
return $libro/título
```



En este ejemplo, se utiliza:

- La cláusula `order by` para ordenar los libros por autor y luego por título. La cláusula `order by` especifica que se deben ordenar los libros **primero por autor y luego por título**.
- La cláusula `return` se utiliza para recuperar el título de cada libro en el orden especificado. Consideremos, ahora, la siguiente consulta:

```
for $estudiante in //estudiante
let $promedio := avg($estudiante/calificaciones/calificacion)
order by $promedio descending
return $estudiante/nombre
```



En este ejemplo, se utiliza:

- La cláusula `let` para calcular el promedio de calificaciones de cada estudiante antes de ordenarlos.
- La cláusula `order by` para ordenar los estudiantes por su promedio de calificaciones de mayor a menor. La cláusula `order by` especifica que se debe ordenar los estudiantes por su promedio de calificaciones, utilizando el operador `descending` para indicar que se deben ordenar de mayor a menor.
- La cláusula `return` se utiliza para recuperar el nombre de cada estudiante en el orden especificado.

`return` #

La cláusula `return` se utiliza para especificar qué información se debe devolver como resultado de una consulta.

La sintaxis básica de la cláusula `return` es la siguiente:

```
for $variable in //elemento
return result
```



donde:

- `$variable` representa el elemento que se va a buscar
- `result` es la información que se desea recuperar.

La cláusula `return` puede contener cualquier tipo de información, desde simples valores numéricos y cadenas de texto hasta estructuras XML complejas. Además, es posible utilizar funciones y expresiones XQuery dentro de la cláusula `return` para realizar cálculos y manipulaciones de datos más avanzados.

Consideremos la siguiente consulta:

```
for $libro in //libro
return $libro/título
```



En este ejemplo, se utiliza la cláusula `return` para recuperar el título de todos los libros de una base de datos XML. La variable `$libro` representa cada elemento de la base de datos que cumple con la condición especificada (en este caso, todos los elementos con la etiqueta `<libro>`). La cláusula `return` se utiliza para recuperar el contenido de la etiqueta `<título>` de cada elemento `$libro`.

Consideremos, ahora, la siguiente consulta:

```
for $estudiante in //estudiante[nombre='Juan']
let $promedio := avg($estudiante/calificaciones/calificación)
return $promedio
```



En este ejemplo, se utiliza la cláusula `return` para recuperar el promedio de calificaciones de un estudiante específico. La variable `$estudiante` representa el elemento de la base de datos que cumple con la condición especificada (en este caso, el elemento con la etiqueta `<estudiante>` cuyo nombre es `Juan`). La cláusula `let` se utiliza para calcular el promedio de calificaciones del estudiante antes de devolverlo como resultado. La cláusula `return` se utiliza para devolver el valor calculado.

Funciones

XQuery incluye **funciones integradas** y permite declarar funciones propias.

Funciones frecuentes

Numéricas

Función	Descripción
<code>floor()</code>	Redondea hacia abajo.
<code>ceiling()</code>	Redondea hacia arriba.
<code>round()</code>	Redondea al valor más próximo.
<code>count()</code>	Cuenta ítems de una secuencia.
<code>min()</code> / <code>max()</code>	Obtiene mínimo o máximo.
<code>avg()</code>	Calcula media.
<code>sum()</code>	Suma total.

Cadenas

Función	Descripción
<code>concat()</code>	Concatena cadenas.
<code>string-length()</code>	Longitud de cadena.
<code>starts-with()</code>	Comprueba prefijo.
<code>ends-with()</code>	Comprueba sufijo.
<code>upper-case()</code>	Convierte a mayúsculas.
<code>lower-case()</code>	Convierte a minúsculas.

Generales

Función	Descripción
---------	-------------

<code>empty()</code>	<code>true</code> si la secuencia está vacía.
<code>exists()</code>	<code>true</code> si la secuencia tiene elementos.
<code>distinct-values()</code>	Elimina duplicados de valores atómicos.
<code>data()</code>	Extrae valor atómico de nodos.

Cuantificadores

Expresión	Descripción
<code>some ... satisfies ...</code>	Existencial: al menos un elemento cumple.
<code>every ... satisfies ...</code>	Universal: todos cumplen.

Definición de funciones

La definición de funciones permite definir operaciones que no están incluidas de forma nativa en XQuery. Además, se utilizan para **modularizar el código**, hacerlo más legible y fácil de mantener. Las funciones también permiten **reutilizar código**, ya que se pueden llamar desde diferentes partes del código. Al igual que en otros lenguajes de programación, las funciones en XQuery también pueden aceptar **argumentos** y devolver **valores**, lo que las hace muy útiles para realizar operaciones específicas y procesar datos.

Sintaxis

Las funciones en XQuery se definen utilizando las palabras clave `declare function` seguida del nombre de la función, sus argumentos y el cuerpo de la función.

```
declare function nombre_funcion($param1 as tipo_dato1, $param2 as tipo_dato2)
as tipo_dato_devuelto
{
    (: Cuerpo de la función :)
}
```

Consideremos la siguiente función XQuery que permite calcular un descuento en un artículo:



```
declare function minPrice($p as xs:decimal?, $d as xs:decimal?)
as xs:decimal?
{
  let $disc := ($p * $d) div 100
  return ($p - $disc)
}
```

En el ejemplo:

- El nombre de la función es `minPrice`
- Recibe dos parámetros: `$p` y `$d`. Ambos parámetros son valores decimales (`xs:decimal`).
- La función devuelve un valor decimal (`xs:decimal`).
- En el cuerpo de la función se realiza una operación con los dos parámetros y se devuelve un valor con `return`.

Para utilizar la función en una consulta XQuery, se debe usar:



```
minPrice(100, 20)
```

Por ejemplo, una consulta completa podría ser la siguiente:



```
for $libro in /biblioteca/libros/libro
return <minPrice>{ minPrice($libro/precio, $libro/descuento) }</minPrice>
```

Operadores

A continuación, se muestran los principales operadores de XQuery, organizados por categorías:

Comparación de valores

Compara dos valores atómicos, y produce error si la secuencia tiene más de un elemento.

Operador	Significado
<code>eq</code>	Igual
<code>ne</code>	Distinto

lt	Menor
le	Menor o igual
gt	Mayor
ge	Mayor o igual

Comparación general (secuencias)

Operador	Significado
= / !=	Igual / distinto
> / >=	Mayor / mayor o igual
< / <=	Menor / menor o igual

Nodos y orden

Operador	Significado
is	Mismo nodo
is not	Distinto nodo
<<	El nodo izquierdo aparece antes en el documento
>>	El nodo izquierdo aparece después en el documento

Lógicos y conjuntos

Operador	Significado
and / or	Combinación lógica

<code>union</code>	Unión de nodos
<code>intersect</code>	Intersección de nodos
<code>except</code>	Diferencia de nodos

Aritméticos

Operador	Significado
<code>+</code> / <code>-</code> / <code>*</code>	Suma, resta, producto
<code>div</code>	División
<code>mod</code>	Resto

Consultas XQuery en ficheros

Las consultas XQuery se pueden escribir directamente en un software que implemente un motor XQuery, el cual permita realizar consultas contra un fichero XML.

Ficheros de consultas

Como alternativa, las consultas también se pueden almacenar en ficheros para su uso posterior o reutilización. Para ello, debemos crear un fichero de texto, escribir la consulta en el fichero y guardarlo con alguna de las siguientes extensiones:

- `.xq`
- `.xqm`
- `.xqy`
- `.xql`
- `.xqu`
- `.xquery`

Por ejemplo, podemos crear un fichero de texto llamado `consulta.xq` que contenga lo siguiente:

```
for $libro in /biblioteca/libros/libro
return $libro/titulo
```



Herramientas

BaseX

BaseX es una **base de datos XML nativa** y motor XPath/XQuery/XSLT.

Para mejorar la **legibilidad** del resultado:

```
declare option output:indent "yes";
```



Saxon

Saxon es un procesador XSLT/XQuery con ediciones *open source* y comerciales.

Ejemplo de ejecución por línea de comandos:

```
java -cp saxon-ee-12.1.jar net.sf.saxon.Query -s:doc.xml -q:query.xq -o:out.xml
```



Parámetros:

- `-s:doc.xml`: documento XML de entrada.
- `-q:query.xq`: consulta XQuery.
- `-o:out.xml`: salida generada.

Estructura típica del paquete:

```
SaxonEE12-1J/
├─ saxon-ee-12.1.jar
├─ saxon-ee-test-12.1.jar
├─ saxon-sql-12.1.jar
├─ doc/
├─ lib/
└─ notices/
```



Fonto (online)

Herramienta web para ejecutar XQuery 3.1 sobre XML.

Pasos básicos:

1. Cargar o pegar el XML en el panel de entrada.
2. Seleccionar modo **XQuery 3.1**.
3. Escribir la expresión en el editor.
4. Revisar el resultado en el panel de salida.

XPath/XQuery Online Tester

También existen testers online para validar expresiones XPath 3.1 y XQuery 3.1 de forma rápida, como [esta](<https://videlibri.de/cgi-bin/xidelcgi>)